

Replicate it. With AI.

The group project, step by step

Advanced Corporate Finance (EBM232A05)

University of Groningen

Where we are going

One project, worth 40% of your grade. Today: exactly what to do, and how a strong team does it.

- The task, and why it looks the way it does.
- The five deliverables, worked end to end on a demo paper.
- Where AI helps, and where you must catch it.

The demo paper is *not* on your menu — it shows the method without handing you an answer.

The project

What you will build

In teams of five, **reproduce the headline table** of a published empirical corporate-finance paper from raw WRDS data — and you must use AI to do it.

- The paper's printed numbers are the **ground truth**.
- You hand in the reproduction, a reproducible code package, a record of your AI use, and a critical extension.
- **40%** of the course grade (the exam is the other 60%), assessed in **four moments**: weeks 39, 41, 42, 43.

Deliverables: **D0 · 10** scope **D1 · 25** reproduction **D2 · 20** package **D3 · 25**
AI-process **D4 · 20** extension

Why a replication?

- A replication has a **ground truth** — the paper's numbers — so it can be graded objectively and at scale.
- You learn the real workflow of empirical corporate finance: *data* → *variables* → *estimation* → *table*.
- Reproduction-as-coursework is an established idea (Berkeley Initiative for Transparency in the Social Sciences, 2021); you are doing it with 2026 tools.

Why AI is required — and graded

- The 2026 research workflow uses AI. We **embrace** it rather than ban it.
- But you remain **fully responsible** for every number, claim, and line of code.
- AI is good at writing code and **bad at judgement** — it silently guesses where a paper is vague (Kohler et al., 2026). Catching that is the graded skill.

Disclosed, documented AI use earns marks. Undisclosed or undocumented use is an integrity violation.

What makes this AI-proof

We grade the things AI **cannot do on your behalf**:

1. **Unique team parameters** — your target sub-table and extension are yours; a generic answer does not fit.
2. **Process, not just product** — real Git history + verbatim AI transcripts, cross-checked against your numbers.
3. **Execution-specific artifacts** — your sample funnel, reconciliation, and sensitivities exist only if you ran it.
4. **An in-tutorial defence** — you explain your own work.
5. **A personal reflection** — in your own words.

AI can do the *work*; it cannot do the *understanding* for the grade.

The demo paper: Fama & French (2001)

Today's demo: "Disappearing Dividends"

Fama and French (2001) ask why fewer firms pay dividends.

- Among NYSE/AMEX/Nasdaq industrials, the share of payers fell 66.5% (1978) → 20.8% (1999).
- Two forces: *changing characteristics* (new firms are small, unprofitable, high-growth) and a *lower propensity to pay*.
- Engine: a **logit** of "pays a dividend" on profitability, growth, and size.

Your team's paper is a **different** paper, drawn from the menu (§A.8). This demo shows the *method* you will apply.

Stage 0 — Scoping

Before you touch data, pin down **exactly** what you reproduce.

- Read the paper **once for the argument**, then again **for the recipe** (sample, variable definitions, the exact table).
- Name the target table and its target cells.
- Write a **variable-definition table** and a data plan.

Output: a short scoping memo, due end of week 38. It is worth **10 points** and it is a **gate** — you must pass it to proceed.

The variable-definition table (demo)

FF2001 demo — build the equivalent for your own paper

Variable	Paper's definition	WRDS construction	Ambiguity?
Payer	pays a dividend this year	$1[DVC > 0]$	common vs. preferred?
Profitability	earnings before interest / assets	$IB(+interest?)/AT$	before/after interest?
Growth	asset growth	$AT/AT_{lag} - 1$	—
Market-to-book	$(A - BE + ME)/A$	$(AT - BE + ME)/AT$	book-equity adj.?
Size	NYSE market-cap percentile	rank vs. NYSE only	vs. NYSE or all?

The four **ambiguities** are the seeds of your Stage-3 dossier — write them down now. You build these exact variables in code in Stage 1.

Exact Compustat items: $BE = CEQ + TXDITC$, $ME = PRCC_F \cdot CSHO$.

- Ask your tool (Mistral) to turn the methods section into a **checklist** of every variable and filter.
- Then **check it against the paper** — the flags above are exactly where the AI papers over a gap.
- Keep those flags: they become your **underspecification catalogue** (D3).

Strong: arrives with the variable-definition table and already knows the four ambiguous choices.

Weak: writes “we will replicate Table 1” and discovers the ambiguities at the end.

Stage 1 — Reproduction

- **D1a — target-cell match** (20): each cell scored by tolerance band (full within $\pm 2\%$, then partial / low / none); sign and significance must match.
- **D1b — sample funnel + reconciliation** (10): account for every material gap.

Small deviations from data vintage are fine **if you explain them**. An unexplained miss is not.

You already know Stata — here we use Python

From your bachelor thesis, the payer logit is a few familiar lines:

```
* Stata -- the way you already know it  
gen payer = dvc > 0  
logit payer prof growth mtb size_pct
```

Ideal for the regression. But **this** project is AI-integrated and reproducible end to end, so we work in **Python**:

- Mistral's **code interpreter runs Python** — you explore *with* the AI in one place; Stata is not in the sandbox.
- The strongest AI coding models (Codestral, Devstral) are **Python-first** — better help, faster catches.
- **One scriptable pipeline** (*wrds* → pandas → statsmodels) is what the package (D2) rewards.

1a — Pull the data: AI writes it, you run it locally

FF2001 demo — build the equivalent for your own paper

No cloud AI can reach WRDS: Mistral **drafts** the pull, **you run it** where WRDS lives (mechanics: Week-37 onboarding). Never paste licensed rows into any AI.

```
import wrds
db = wrds.Connection()                # run locally; your WRDS login
funda = db.raw_sql("""
    select gvkey, datadate, sic, at, dvc, dvp, ib, ceq, txditc, prcc_f, csho
    from comp.funda
    where indfmt='INDL' and datafmt='STD' and popsrc='D' and consol='C'
    and datadate between '1963-01-01' and '1999-12-31' and at > 0
""")
fin = funda.sic.between('6000','6999')    # do exclusions in code,
util = funda.sic.between('4900','4949')    # so the funnel is auditable
df = funda[~fin & ~util].copy()
```

Build the sample funnel

FF2001 demo — build the equivalent for your own paper

Record the row count **after each filter** — from *your* pull on *your* date.

Step	Firm-years
Raw <i>comp. funda</i> industrials 1963–1999, $AT > 0$	$\approx 250,000$ (illustrative)
– financials (SIC 6000–6999)	...
– utilities (SIC 4900–4949)	...
– missing any regression variable	...
Final estimation sample	N (vs. the paper's N)

These counts are **yours** and un-guessable — which is exactly why they are graded.

1b — Build variables & estimate

FF2001 demo — build the equivalent for your own paper

```
df['payer'] = (df['dvc'] > 0).astype(int)
df['prof'] = df['ib'] / df['at'] # before/after interest? -> catch
df['growth'] = df.sort_values(['gvkey', 'datadate']).groupby('gvkey')['at'].pct_change()
be = df['ceq'] + df['txditc'].fillna(0) # one defensible book-equity choice
df['mtb'] = (df['at'] - be + df['prcc_f']*df['csho']) / df['at']

import statsmodels.formula.api as smf, numpy as np
coefs = [smf.logit('payer ~ prof + growth + mtb + size_pct', data=g).fit(dis=0).params
         for _, g in df.groupby(df.datadate.dt.year)] # year by year (Fama-MacBeth)
avg = np.mean(coefs, axis=0) # the target coefficients
```

Why year-by-year, then average? Each year gets equal weight and cross-year residual correlation does not bias the average — the Fama–MacBeth logic.

1c — Reconcile every miss

For each cell off by more than the band, name the **root cause** (Kohler et al., 2026)
— *guess the category before you read on: data vintage · paper underspecification · your choice · suspected paper error.*

- Payer% 1978 = 66.9 vs. 66.5 → *data vintage* (Compustat revised); in band.
- Profitability coef. 30% low → *underspecification*: FF use earnings **before** interest; refit with *IB+XINT* → in band.
- *N* 4% below → *your choice*: you required non-missing *mtb*. Documented.

Strong: shows the funnel and proves each gap by re-running the alternative.

Weak: “small differences are due to data.”

Stage 2 — Reproducible package

Hand in a **Git repository with real history**, not a zip.

```
readme.md          <- what/how + data-availability statement
run_all.py         <- one command regenerates every number in /output
environment.yml     <- pinned packages + seed
data/raw-metadata/ <- WRDS query .sql + pull date   (NO raw extracts)
data/derived/analysis.parquet
code/ 01_pull_wrds.py 02_build_vars.py 03_estimate.py 04_tables.py
output/ table1_logit.csv funnel.csv payer_share.png run.log
```

The grader runs `run_all.py` and checks the regenerated numbers equal your submitted ones — the anti-hardcoding check.

Real commit history (it is inspected)

FF2001 demo — build the equivalent for your own paper

```
a1c  scaffold repo + readme
b4f  01_pull: funda pull + SIC exclusions; funnel prints
9d2  02_build: payer, profitability, growth, mtb
7e8  fix: profitability should be pre-interest (IB+XINT)      <- a catch
3aa  02_build: NYSE-only size percentile (was full-sample)    <- a catch
c10  03_estimate: year-by-year logit + FM average
f77  04_tables: Table 1 + funnel + payer-share figure
```

- Genuine, incremental cadence scores; a single “add everything” commit scores **zero**.
- Ship **query specs + derived data**, never raw WRDS extracts (licence).

New to Git? Make many small, meaningful commits as you work — not one big dump at the end.

Stage 3 — AI-process dossier

Graded on **real artifacts**, not your summary of them. Three things earn the marks:

1. **Underspecification catalogue** (≥ 3): AI's default vs. your choice vs. a sensitivity.
2. **Documented catches** (≥ 3): where the AI was **wrong**, how you caught it, the fix — each cited to a transcript line **and** a commit.
3. **A fresh-context “Referee-2” audit**: a new AI session reviews your near-final code.

Attach the **verbatim transcripts**. No transcripts $\Rightarrow$ the dossier is unverifiable prose and scores near zero.

A concrete catch

FF2001 demo — build the equivalent for your own paper

Size variable. The paper's size is a percentile in the **NYSE** distribution — Mistral ranked over the **whole sample**.

AI's default — wrong universe:

```
df['size_pct'] = df['mktcap'].rank(pct=True)
```

Your fix — NYSE breakpoints only:

```
df['size_pct'] = nyse_percentile(df, 'mktcap') # rank within NYSE firms
```

Caught: payer-rate-by-size looked too flat. **Sensitivity:** size coef. 0.31 → 0.52. **Evidence:** *chat-02.md* L40–71; commit 3aa.

What check, before trusting it, would have caught this?

The fresh-context “Referee-2” audit

FF2001 demo — build the equivalent for your own paper

Before your final run, open a **new** AI session (no prior context) and ask it to audit your near-final code.

```
# fresh session, pasted 03_estimate.py:  
"Check this for look-ahead bias and sample-alignment bugs."
```

- **It flagged:** *growth* used *AT* before dropping firm-years with missing lagged *AT* — inflating early-year *N*.
- **You did:** added the drop, re-ran; the early-year counts corrected.
- **Evidence:** transcript *referee2.md*; commit *d19*.

A fresh context catches what your primed, tired session no longer sees.

One page, personal and specific. An ideal excerpt:

“The line I spent longest on was book equity for market-to-book. Mistral used CEQ alone; the paper adds deferred taxes. I ran both and the market-to-book (mtb) coefficient changed significance at the margin. The AI hands you a defensible-looking default that quietly changes your result — I now diff every definition against the paper before trusting it.”

- Generic reflection that could attach to any project scores low.
- Must be consistent with the team dossier — and with your **defence**.

Stage 4 — Critical extension

Take your **assigned extension axis** one defensible step beyond the paper. Demo:

- Extend Fama–French to **2024**: did the disappearance reverse?
- Dividends partly *reappeared* and *repurchases* grew — payout $\neq$ dividends.

Mandatory three-perspective hook — which view does your evidence favour?

- *Shareholder value / life-cycle* (DeAngelo et al., 2006): mature, cash-rich firms distribute.
- *Principal-agent* (Jensen, 1986): dividends disgorge free cash flow, disciplining managers.
- *Behavioural* — catering (Baker and Wurgler, 2004): firms pay when payers earn a premium.

Banded: correct variation + interpretation ≈ 14 ; a genuine, theory-linked contribution $\rightarrow 20$.

Stage 5 – The defence

A roughly 5-minute walkthrough of **your** repository, then 1–2 questions drawn at random:

- “Show where your funnel drops from X to Y, and why.”
- “Change this winsorization live — which target cell moves most?”
- “Explain commit 3aa.”

It does not add points — it **gates** them: a team that cannot explain its submission has **its** D1b/D2b/D3a/D4 capped at 50%.

You cannot defend what you did not do. Prepare = know your funnel, your ambiguities, your two best catches.

Wrap-up

Ideal pacing (weeks 36–43)

Week	Ideal state
36	team + paper ranked; read the paper twice
37	variable-definition table drafted; WRDS access working
38	scoping memo submitted; first pull + funnel running
39	M1 — memo returned + gate; variables built; catches logged <i>as they happen</i>
40	Career Week: target cells brought into band <i>around</i> your events
41	M2 — full table reconciled and submitted (D1)
42	M3 — package, dossier, extension (D2, D3a, D4)
43	M4 — individual reflections (D3b); defence

The one habit that separates strong teams: keep the dossier **as you work**, not at the end.

How it maps to the rubric

Stage	Deliverable	Moment	Points
0 Scope	D0	M1 (wk 39)	10
1 Pull, estimate, reconcile	D1	M2 (wk 41)	25
2 Package (re-run + history)	D2	M3 (wk 42)	20
3 Dossier	D3a	M3 (wk 42)	15
4 Extension	D4	M3 (wk 42)	20
5 Reflection	D3b	M4 (wk 43)	10
6 Defence	—	M4 (wk 43)	<i>gates D1b/D2b/D3a/D4</i>

100 points → 40 % of the course grade. $\approx$ 30 of them are scripted; most of the rest are rubric-graded with a human check.

Do: run the pull locally; build the funnel as you filter; log every catch with a transcript line and a commit; keep the dossier live; be ready to defend your own code.

Don't: paste licensed WRDS data into any AI; submit a single “add everything” commit; write a reflection that could belong to any project; reconstruct the dossier the night before.

AI is your documented collaborator — not a black box you hide behind.

- Baker, M. and J. Wurgler (2004). **“A Catering Theory of Dividends”**. In: *The Journal of Finance* 59.3, pp. 1125–1165.
- Berkeley Initiative for Transparency in the Social Sciences (2021). **Accelerating Computational Reproducibility in Economics (ACRe) Guide and the Social Science Reproduction Platform**. <https://bitss.github.io/ACRe/>.
- DeAngelo, H., L. DeAngelo, and R. M. Stulz (2006). **“Dividend Policy and the Earned/Contributed Capital Mix: A Test of the Life-Cycle Theory”**. In: *Journal of Financial Economics* 81.2, pp. 227–254.
- Fama, E. F. and K. R. French (2001). **“Disappearing Dividends: Changing Firm Characteristics or Lower Propensity to Pay?”** In: *Journal of Financial Economics* 60.1, pp. 3–43.

- Jensen, M. C. (1986). **“Agency Costs of Free Cash Flow, Corporate Finance, and Takeovers”**. In: *American Economic Review* 76.2, pp. 323–329.
- Kohler, B. et al. (2026). **“Read the Paper, Write the Code: Agentic Reproduction of Social-Science Results”**. Working paper, ETH Zurich.